

Reproduzierbarer Ablauf: YOLOv8s Benchmark auf PHYTEC phyCORE-i.MX95 mit Manjaro-Host

Dieses Dokument beschreibt einen **komplett neu aufgesetzten**, auf **aktuelles Manjaro** zugeschnittenen Ablauf, um einen reproduzierbaren YOLOv8s-Benchmark für den PHYTEC phyCORE-i.MX95 vorzubereiten. Ziel ist, dass du das Dokument direkt an eine andere KI oder einen Kollegen weitergeben kannst, damit die Arbeit ohne viel Rückfrage erledigt werden kann. PHYTEC stellt für den i.MX95 den BSP-Download über das `phyLinux`-Skript bereit und verlangt vorher die Auswahl des passenden Machine-Namens.[web:16][web:14]

1. Ziel

Zu liefern sind für YOLOv8s auf COCO val2017:

- mAP50
- mAP50-95
- FPS
- Latenz
- klare Aussage, ob NMS und Memory Transfers in den Zeiten enthalten sind.[web:26][web:35]

Zusätzlich muss die Zielforderung bewertet werden, ob ein **custom YOLOv8s** bei 1024 x 1024, Batch 1, mit **mindestens 1 FPS sustained** inklusive Preprocessing und Postprocessing unter realistischen thermischen Bedingungen auf dem phyCORE-i.MX95 möglich ist.[web:16]

2. Grundprinzip des Ablaufs

Der Ablauf ist in zwei Teile getrennt:

1. **Host unter Manjaro:** Dataset laden, YOLOv8s validieren, ONNX exportieren, Dateien zum Target übertragen.[web:30][web:26]
2. **Target auf i.MX95:** Inferenzpipeline ausführen, COCO-Detections schreiben, COCO-Eval rechnen, Timing und Thermal dokumentieren.[web:16]

Diese Trennung ist wichtig, weil das Modell-Handling und die Datensatzvorbereitung auf dem Host bequemer sind, während die eigentliche Zielmessung auf der i.MX95-Hardware stattfinden muss.[web:16][web:14]

3. Projektstruktur

```
imx95-yolov8-benchmark/  
├─ README.md  
├─ requirements-host.txt  
├─ requirements-target.txt
```

```
├─ env.sh
├─ scripts/
│   ├── 00_host_setup_manjaro.sh
│   ├── 01_download_coco.sh
│   ├── 02_download_model.sh
│   ├── 03_validate_pytorch.sh
│   ├── 04_export_onnx.sh
│   ├── 05_check_onnx.sh
│   ├── 06_transfer_to_target.sh
│   ├── 07_target_setup.sh
│   ├── 08_run_inference_imx95.py
│   ├── 09_eval_coco.py
│   ├── 10_collect_thermal.sh
│   └─ 11_generate_report.sh
├─ data/
│   └─ coco/
├─ models/
├─ results/
└─ logs/
```

4. Host-Voraussetzungen unter Manjaro

Für Manjaro solltest du zuerst die üblichen Build- und Python-Werkzeuge per `pacman` installieren. Ultralytics empfiehlt eine saubere Python-Umgebung und die Installation per `pip install -U ultralytics`; zusätzlich brauchst du ONNX Runtime und COCO-Tools für Export und Evaluation.[web:30][web:29][web:34]

4.1 Systempakete installieren

Datei `scripts/00_host_setup_manjaro.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

sudo pacman -Syu --needed \
    base-devel \
    git \
    wget \
    unzip \
    curl \
    rsync \
    python \
    python-pip \
    python-virtualenv

python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip wheel setuptools
pip install -r requirements-host.txt

yolo checks
python - <<'PY'
from ultralytics import YOLO
```

```
print('Ultralytics import OK')
PY
```

4.2 Python-Abhängigkeiten

Datei `requirements-host.txt`:

```
ultralytics
onnx
onnxruntime
onnxsim
pycocotools
opencv-python
numpy
pandas
pyyaml
```

Hinweis für Manjaro: `pycocotools` kann je nach Python- und Toolchain-Stand einen Compiler benötigen; genau deshalb werden `base-devel` und eine frische virtuelle Umgebung empfohlen.[web:28][web:31]

5. COCO val2017 herunterladen

COCO val2017 und die Annotationen können direkt von `images.cocodataset.org` heruntergeladen werden; für die Auswertung wird `instances_val2017.json` benötigt.[web:21][web:18]

Datei `scripts/01_download_coco.sh`:

```
#!/usr/bin/env bash
set -eou pipefail
mkdir -p data/coco
cd data/coco

wget -c http://images.cocodataset.org/zips/val2017.zip
wget -c http://images.cocodataset.org/annotations/annotations_trainval2017.zip

unzip -o val2017.zip
unzip -o annotations_trainval2017.zip

test -d val2017
test -f annotations/instances_val2017.json
```

6. YOLOv8s laden und Referenzvalidierung ausführen

Ultralytics stellt das `yolo-CLI` bereit und dokumentiert Validierung und Export als Standardweg für Linux/Python-Setups.[web:30][web:26]

Datei `scripts/02_download_model.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
source .venv/bin/activate
mkdir -p models
python - <<'PY'
from ultralytics import YOLO
YOLO('yolov8s.pt')
print('Downloaded yolov8s.pt')
PY
find ~/.cache -name 'yolov8s.pt' -print | head -n 5
```

Date: data/coco/coco_val2017.yaml:

```
path: ./data/coco
val: val2017
names:
  0: person
  1: bicycle
  2: car
  3: motorcycle
  4: airplane
  5: bus
  6: train
  7: truck
  8: boat
  9: traffic light
  10: fire hydrant
  11: stop sign
  12: parking meter
  13: bench
  14: bird
  15: cat
  16: dog
  17: horse
  18: sheep
  19: cow
  20: elephant
  21: bear
  22: zebra
  23: giraffe
  24: backpack
  25: umbrella
  26: handbag
  27: tie
  28: suitcase
  29: frisbee
  30: skis
  31: snowboard
  32: sports ball
  33: kite
  34: baseball bat
  35: baseball glove
  36: skateboard
  37: surfboard
```

38: tennis racket
39: bottle
40: wine glass
41: cup
42: fork
43: knife
44: spoon
45: bowl
46: banana
47: apple
48: sandwich
49: orange
50: broccoli
51: carrot
52: hot dog
53: pizza
54: donut
55: cake
56: chair
57: couch
58: potted plant
59: bed
60: dining table
61: toilet
62: tv
63: laptop
64: mouse
65: remote
66: keyboard
67: cell phone
68: microwave
69: oven
70: toaster
71: sink
72: refrigerator
73: book
74: clock
75: vase
76: scissors
77: teddy bear
78: hair drier
79: toothbrush

Date: scripts/03_validate_pytorch.sh:

```
#!/usr/bin/env bash
set -euo pipefail
source .venv/bin/activate
mkdir -p results logs

yolo detect val \
  model=yolov8s.pt \
  data=data/coco/coco_val2017.yaml \
  imgsz=1024 \
  batch=1 \
```

```
device=cpu \  
project=results \  
name=val_pytorch_1024 | tee logs/03_validate_pytorch.log
```

Dieser Schritt erzeugt deine **Host-Referenz** für mAP auf COCO val2017, bevor du das Modell in ONNX oder eine NPU-spezifische Form bringst.[web:26][web:30]

7. ONNX exportieren

Ultralytics dokumentiert den Export nach ONNX sowohl per CLI als auch per Python; dabei sind Parameter wie `imgsz`, `batch`, `simplify`, `opset` und `nms` relevant.[web:26][web:35]

Datei `scripts/04_export_onnx.sh`:

```
#!/usr/bin/env bash  
set -euo pipefail  
source .venv/bin/activate  
mkdir -p models logs  
python - <<'PY'  
from ultralytics import YOLO  
model = YOLO('yolov8s.pt')  
out = model.export(  
    format='onnx',  
    imgsz=(1024, 1024),  
    batch=1,  
    simplify=True,  
    opset=13,  
    nms=False  
)  
print(out)  
PY  
  
find . -maxdepth 2 -name '*.onnx' -print | tee logs/04_export_onnx_files.log  
cp yolov8s.onnx models/yolov8s_1024.onnx 2>/dev/null || true
```

Warum `nms=False`? Weil du damit NMS außerhalb des Graphen hältst und später sauber angeben kannst, ob NMS in der finalen Timing-Zahl enthalten ist; genau diese Transparenz wird gefordert.[web:35][web:26]

Optional zweiter Exportpfad:

```
source .venv/bin/activate  
yolo export model=yolov8s.pt format=onnx imgsz=1024,1024 batch=1 simplify=True opse-
```

Dieser zweite Pfad ist nur sinnvoll, wenn die Zielruntime mit exportiertem NMS stabil umgehen kann; laut Exporter-Referenz ist `nms=True` ein expliziter Spezialfall im Exportprozess.[web:35]

8. ONNX prüfen

Datei `scripts/05_check_onnx.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
source .venv/bin/activate
python - <<'PY'
import onnx
m = onnx.load('models/yolov8s_1024.onnx')
onnx.checker.check_model(m)
print('ONNX model is valid')
print('IR version:', m.ir_version)
print('Number of graph nodes:', len(m.graph.node))
PY
```

9. PHYTEC BSP beziehen

PHYTEC beschreibt für iMX95 den BSP-Bezug über das `phyLinux`-Skript und weist darauf hin, dass vorab der passende Machine-Name identifiziert werden muss.[web:16] [web:14]

Host-seitiges Beispiel unter Manjaro:

```
mkdir -p ~/yocto
cd ~/yocto
wget https://download.phytec.de/Software/Linux/Yocto/Tools/phyLinux
chmod +x phyLinux
./phyLinux init
```

Zusätzliche Pflichtaufgabe für die ausführende KI:

- exakten PHYTEC Machine-Namen für dein Board bestimmen
- korrekte iMX95 BSP-Version auswählen
- prüfen, ob das BSP schon eine passende NPU-Runtime mitbringt
- prüfen, ob direkter ONNX-Import möglich ist oder ein NXP-spezifischer Konverter nötig wird

10. Transfer auf das Target

Datei `env.sh`:

```
export TARGET_IP=192.168.3.20
export TARGET_USER=root
export TARGET_DIR=/opt/imx95-yolov8-benchmark
```

Datei `scripts/06_transfer_to_target.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
source env.sh
ssh ${TARGET_USER}@${TARGET_IP} "mkdir -p ${TARGET_DIR}/models ${TARGET_DIR}/data/coco
scp models/yolov8s_1024.onnx ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/models/
scp -r data/coco/val2017 ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/data/coco/
scp data/coco/annotations/instances_val2017.json ${TARGET_USER}@${TARGET_IP}:${TARGET_
scp requirements-target.txt ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/
scp scripts/08_run_inference_imx95.py ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/scrip
scp scripts/09_eval_coco.py ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/scripts/
scp scripts/10_collect_thermal.sh ${TARGET_USER}@${TARGET_IP}:${TARGET_DIR}/scripts/
```

11. Target-Setup auf dem i.MX95

Datei requirements-target.txt:

```
numpy
opencv-python
pycocotools
onnxruntime
pandas
```

Datei scripts/07_target_setup.sh:

```
#!/usr/bin/env bash
set -euo pipefail
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip wheel setuptools
pip install -r requirements-target.txt
```

Wichtig: onnxruntime auf dem Target ist zunächst nur der Referenzpfad. Für die finale Aussage zum i.MX95 musst du die reale NPU-Toolchain des BSP verifizieren, weil die Hardwarebeschleunigung nicht automatisch aus einem generischen ONNX-Runtime-Setup folgt.[web:34][web:16]

12. Inferenzskript auf dem Target

Datei scripts/08_run_inference_imx95.py:

```
#!/usr/bin/env python3
import json
import time
from pathlib import Path

import cv2
import numpy as np
import onnxruntime as ort
import pandas as pd
from pycocotools.coco import COCO
```



```

ROOT = Path('/opt/imx95-yolov8-benchmark')
IMAGES = ROOT / 'data/coco/val2017'
ANN = ROOT / 'data/coco/instances_val2017.json'
MODEL = ROOT / 'models/yolov8s_1024.onnx'
RESULTS = ROOT / 'results'
RESULTS.mkdir(parents=True, exist_ok=True)

IMG_SIZE = 1024
CONF_THRES = 0.25
IOU_THRES = 0.7

coco = COCO(str(ANN))
img_ids = coco.getImgIds()
imgs = coco.loadImgs(img_ids)

sess = ort.InferenceSession(str(MODEL), providers=['CPUExecutionProvider'])
input_name = sess.get_inputs()[0].name

def letterbox(im, new_shape=(1024, 1024), color=(114,114,114)):
    shape = im.shape[:2]
    r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
    new_unpad = (int(round(shape[1] * r)), int(round(shape[0] * r)))
    dw, dh = new_shape[1] - new_unpad[0], new_shape[0] - new_unpad[1]
    dw /= 2
    dh /= 2
    if shape[::-1] != new_unpad:
        im = cv2.resize(im, new_unpad, interpolation=cv2.INTER_LINEAR)
    top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
    left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
    im = cv2.copyMakeBorder(im, top, bottom, left, right, cv2.BORDER_CONSTANT, value=color)
    return im, r, (dw, dh)

def preprocess(img_path):
    im = cv2.imread(str(img_path))
    assert im is not None, f'Failed to read {img_path}'
    im0 = im.copy()
    im, ratio, dw, dh = letterbox(im, (IMG_SIZE, IMG_SIZE))
    im = cv2.cvtColor(im, cv2.COLOR_BGR2RGB)
    im = im.astype(np.float32) / 255.0
    im = np.transpose(im, (2, 0, 1))[None]
    return im, im0, ratio, dw, dh

def nms_numpy(bboxes, scores, iou_thres):
    x1 = bboxes[:, 0]
    y1 = bboxes[:, 1]
    x2 = bboxes[:, 2]
    y2 = bboxes[:, 3]
    areas = (x2 - x1) * (y2 - y1)
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]

```

```

        keep.append(i)
        xx1 = np.maximum(x1[i], x1[order[1:]])
        yy1 = np.maximum(y1[i], y1[order[1:]])
        xx2 = np.minimum(x2[i], x2[order[1:]])
        yy2 = np.minimum(y2[i], y2[order[1:]])
        w = np.maximum(0.0, xx2 - xx1)
        h = np.maximum(0.0, yy2 - yy1)
        inter = w * h
        union = areas[i] + areas[order[1:]] - inter
        iou = inter / np.maximum(union, 1e-6)
        inds = np.where(iou <= iou_thres)[0]
        order = order[inds + 1]
    return keep

```

```

def postprocess(pred, im0_shape, ratio, dwdh):
    pred = np.squeeze(pred).T
    boxes = pred[:, :4]
    scores = pred[:, 4:]
    cls_ids = np.argmax(scores, axis=1)
    cls_scores = scores[np.arange(scores.shape[0]), cls_ids]
    mask = cls_scores > CONF_THRES
    boxes = boxes[mask]
    cls_ids = cls_ids[mask]
    cls_scores = cls_scores[mask]
    if len(boxes) == 0:
        return []
    boxes_xyxy = np.empty_like(boxes)
    boxes_xyxy[:, 0] = boxes[:, 0] - boxes[:, 2] / 2
    boxes_xyxy[:, 1] = boxes[:, 1] - boxes[:, 3] / 2
    boxes_xyxy[:, 2] = boxes[:, 0] + boxes[:, 2] / 2
    boxes_xyxy[:, 3] = boxes[:, 1] + boxes[:, 3] / 2
    dw, dh = dwdh
    boxes_xyxy[:, [0, 2]] -= dw
    boxes_xyxy[:, [1, 3]] -= dh
    boxes_xyxy /= ratio
    boxes_xyxy[:, [0, 2]] = boxes_xyxy[:, [0, 2]].clip(0, im0_shape[1])
    boxes_xyxy[:, [1, 3]] = boxes_xyxy[:, [1, 3]].clip(0, im0_shape[0])
    dets = []
    for c in np.unique(cls_ids):
        idx = np.where(cls_ids == c)[0]
        keep = nms_numpy(boxes_xyxy[idx], cls_scores[idx], IOU_THRES)
        for k in keep:
            j = idx[k]
            x1, y1, x2, y2 = boxes_xyxy[j]
            dets.append({
                'category_id': int(c) + 1,
                'bbox_xyxy': [float(x1), float(y1), float(x2), float(y2)],
                'score': float(cls_scores[j])
            })
    return dets

```

```

def xyxy_to_coco_bbox(xyxy):
    x1, y1, x2, y2 = xyxy
    return [x1, y1, x2 - x1, y2 - y1]

```

```

all_dets = []
timing_rows = []

for meta in imgs:
    img_path = IMAGES / meta['file_name']

    t0 = time.perf_counter()
    inp, im0, ratio, dwdh = preprocess(img_path)
    t1 = time.perf_counter()

    t2 = time.perf_counter()
    pred = sess.run(None, {input_name: inp})[0]
    t3 = time.perf_counter()

    dets = postprocess(pred, im0.shape, ratio, dwdh)
    t4 = time.perf_counter()

    for d in dets:
        all_dets.append({
            'image_id': meta['id'],
            'category_id': d['category_id'],
            'bbox': xyxy_to_coco_bbox(d['bbox_xyxy']),
            'score': d['score']
        })

    timing_rows.append({
        'image_id': meta['id'],
        'preprocess_ms': (t1 - t0) * 1000.0,
        'inference_ms': (t3 - t2) * 1000.0,
        'postprocess_ms': (t4 - t3) * 1000.0,
        'total_ms': (t4 - t0) * 1000.0,
        'nms_included_in_postprocess': True,
        'memory_transfer_included': False
    })

with open(RESULTS / 'coco_detections_imx95.json', 'w') as f:
    json.dump(all_dets, f)

pd.DataFrame(timing_rows).to_csv(RESULTS / 'timing_breakdown.csv', index=False)

avg_total_ms = float(np.mean([r['total_ms'] for r in timing_rows]))
fps = 1000.0 / avg_total_ms if avg_total_ms > 0 else 0.0
summary = {
    'dataset': 'COCO val2017',
    'imgsz': 1024,
    'batch': 1,
    'avg_latency_ms': avg_total_ms,
    'fps_from_avg_latency': fps,
    'timing_definition': {
        'includes_preprocess': True,
        'includes_inference': True,
        'includes_postprocess': True,
        'includes_nms': True,
        'includes_memory_transfers': False,
        'notes': 'Reference ONNX path on target. Replace with NPU timing path for fi

```

```

    }
}
with open(RESULTS / 'metrics_imx95_timing_only.json', 'w') as f:
    json.dump(summary, f, indent=2)

```

13. COCO-Eval-Skript

Dateiscripts/09_eval_coco.py:

```

#!/usr/bin/env python3
import json
from pathlib import Path
from pycocotools.coco import COCO
from pycocotools.cocoeval import COCOeval

ROOT = Path('/opt/imx95-yolov8-benchmark')
ann_file = ROOT / 'data/coco/instances_val2017.json'
det_file = ROOT / 'results/coco_detections_imx95.json'
out_file = ROOT / 'results/metrics_imx95.json'

coco_gt = COCO(str(ann_file))
coco_dt = coco_gt.loadRes(str(det_file))

eval = COCOeval(coco_gt, coco_dt, 'bbox')
eval.evaluate()
eval.accumulate()
eval.summarize()

stats = {
    'mAP50_95': float(eval.stats[0]),
    'mAP50': float(eval.stats[1]),
    'mAP75': float(eval.stats[2]),
    'AP_small': float(eval.stats[3]),
    'AP_medium': float(eval.stats[4]),
    'AP_large': float(eval.stats[5])
}
with open(out_file, 'w') as f:
    json.dump(stats, f, indent=2)

```

Die COCO API ist der Standardweg für die Auswertung von Detection-Ergebnissen gegen instances_val2017.json.[web:18][web:21]

14. Thermal- und Umgebungsdaten sammeln

Dateiscripts/10_collect_thermal.sh:

```

#!/usr/bin/env bash
set -euo pipefail
OUT=/opt/imx95-yolov8-benchmark/results/environment.txt
{
    date
    uname -a
    echo

```

```

echo '== CPU info =='
cat /proc/cpuinfo || true
echo
echo '== Memory =='
free -h || true
echo
echo '== Thermal zones =='
for z in /sys/class/thermal/thermal_zone*; do
    echo "ZONE: $z"
    cat "$z/type" 2>/dev/null || true
    cat "$z/temp" 2>/dev/null || true
    echo
done
} > "$OUT"

```

15. Ausführungsreihenfolge

Auf dem Manjaro-Host

```

cd imx95-yolov8-benchmark
chmod +x scripts/*.sh
./scripts/00_host_setup_manjaro.sh
./scripts/01_download_coco.sh
./scripts/02_download_model.sh
./scripts/03_validate_pytorch.sh
./scripts/04_export_onnx.sh
./scripts/05_check_onnx.sh
./scripts/06_transfer_to_target.sh

```

Auf dem i.MX95-Target

```

cd /opt/imx95-yolov8-benchmark
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-target.txt
python3 scripts/08_run_inference_imx95.py
python3 scripts/09_eval_coco.py
bash scripts/10_collect_thermal.sh

```

16. Report-Template

Dateireport.md:

```

# Benchmark Report: YOLOv8s on PHYTEC phyCORE-i.MX95

## System
- Host: Manjaro Linux
- Board: PHYTEC phyCORE-i.MX95
- BSP: <exakte PHYTEC/NXP Version>
- Runtime: <onnxruntime / eIQ / vendor runtime>

```

```

- Cooling: <definieren>
- Ambient: <definieren>

## Model
- Model: YOLOv8s
- Dataset: COCO val2017
- Input: 1024x1024
- Batch: 1
- Export: ONNX, opset 13, simplify=True, nms=False

## Accuracy
- mAP50: <Wert>
- mAP50-95: <Wert>

## Performance
- Avg Latency: <Wert> ms
- FPS: <Wert>

## Timing Definition
- Includes preprocessing: Yes/No
- Includes inference: Yes/No
- Includes postprocessing: Yes/No
- Includes NMS: Yes/No
- Includes memory transfers: Yes/No
- Timing start: <definieren>
- Timing end: <definieren>

## Thermal
- Duration: <z. B. 10 min>
- SoC temperature range: <Werte>
- Any throttling observed: Yes/No

```

17. Direkt nutzbarer Auftrag an eine andere KI

Diesen Block kannst du fast unverändert weitergeben:

1. Arbeite exakt nach diesem Manjaro-basierten Ablauf.
2. Installiere die Host-Abhängigkeiten per pacman und Python-Venv.[web:30][web:28]
3. Lade COCO val2017 und instances_val2017.json herunter.[web:21][web:18]
4. Validiere YOLOv8s mit Ultralytics auf dem Host bei `imgsz=1024, batch=1`. [web:26]
5. Exportiere das Modell nach ONNX mit `simplify=True, opset=13, nms=False`. [web:35][web:26]
6. Richte den PHYTEC iMX95 BSP-Zugang über phyLinux ein und identifiziere den korrekten Machine-Namen. [web:16][web:14]
7. Prüfe die verfügbare NPU-Runtime im BSP und ersetze den ONNX-Referenzpfad, falls eine native NPU-Toolchain genutzt werden kann. [web:16]
8. Führe die Inferenz auf dem Board über alle COCO-val2017-Bilder aus und speichere COCO-konforme Detection-Ergebnisse.
9. Berechne mAP50 und mAP50-95 mit `pycocotools`. [web:18]

10. Dokumentiere klar, ob NMS und Memory-Transfers in FPS/Latenz enthalten sind; ohne diese Aussage ist der Benchmark unvollständig.[web:35]
11. Ergänze einen Sustained-Test unter dokumentierten thermischen Randbedingungen.
12. Erzeuge einen finalen Report anhand des Templates.

18. Wichtige Einschränkung

Dieses Dokument ist absichtlich **Host-seitig auf Manjaro korrigiert**, aber die endgültige NPU-Integration auf dem iMX95 bleibt trotzdem BSP- und Runtime-abhängig. Genau deshalb muss die ausführende KI nicht nur die Skripte ausführen, sondern auch die reale PHYTEC/NXP-Softwarelage des konkreten Boards validieren, bevor die finalen Kundenzahlen als belastbar gelten.[web:16][web:14]